

Software Engineering II – Coding Standards Lab 2026

Escuela Superior Politécnica del Litoral – Faculty of Electrical and Computer Engineering

Objectives

- Use a code quality tool to detect and enforce coding standards.
- Refactor and complete a faulty base program so it meets a given set of functional requirements.
- Obtain HTML reports of the issues detected in the code, before and after your changes.

Requirements

- Visual Studio Code
- Python (Section A) or Java 8+ JDK (Section B)
- Eclipse IDE for Java Developers (optional, Section B)
- Git and a GitHub or GitLab account

Introduction

Code conventions and best practices keep a codebase consistent and readable. Following them improves code quality, eases collaboration and reduces errors. Enforcing them by hand is tedious, so we use automated tools.

Linting tools analyze source code and flag deviations from coding standards. They check class design, method design, layout and formatting, and can produce project-wide reports. Typical check categories include:

- Annotations, block checks, class design, coding
- Headers, imports, Javadoc comments, metrics
- Modifiers, naming conventions, size violations, whitespace

In this lab you receive a **base program that is intentionally poorly written and full of errors**. Your job is to use a coding standards tool to find the problems, then rewrite the program so it follows good practices **and** meets all the functional requirements listed in the *Functional Requirements* section. The lab offers two versions, one per language: Section A (Python) and Section B (Java). You only complete one.

Activity

Step 1 – Choose the programming language

1. Choose **Python (Section A)** or **Java (Section B)**, based on your familiarity or interest.
2. Read the base code of your section and the *Functional Requirements* section carefully before you start. The base code is intentionally faulty: it has syntax errors, runtime errors, bad naming and missing features. Do not fix anything yet.

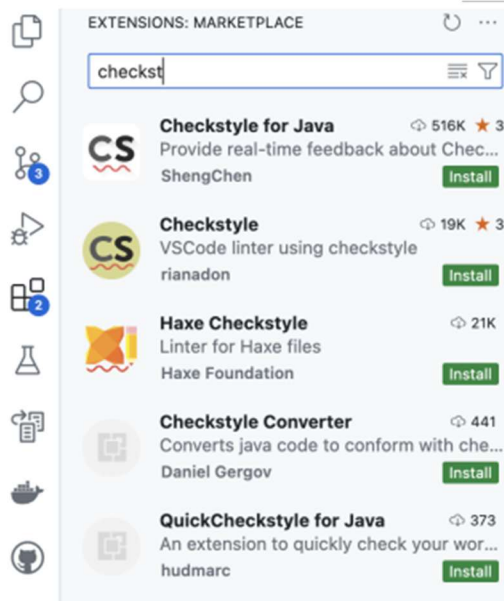
Step 2 – Create a repository with the base code

1. Create a new **public** repository on GitHub or GitLab for your chosen language. Give it a meaningful name and initialize it with a README file.
2. Clone the repository and open it in Visual Studio Code.
3. Create the source file(s) and copy the base code of your section **exactly as given**. Commit and push this first version.

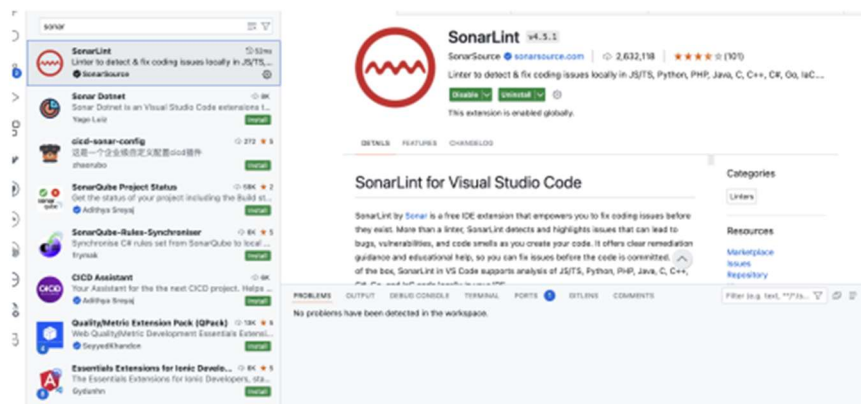
Challenge (5 pts): Create a GitHub Actions workflow that runs your coding standards tool on every pull request to the `main` branch.

Step 3 – Run the code standards tool on the base code (initial report)

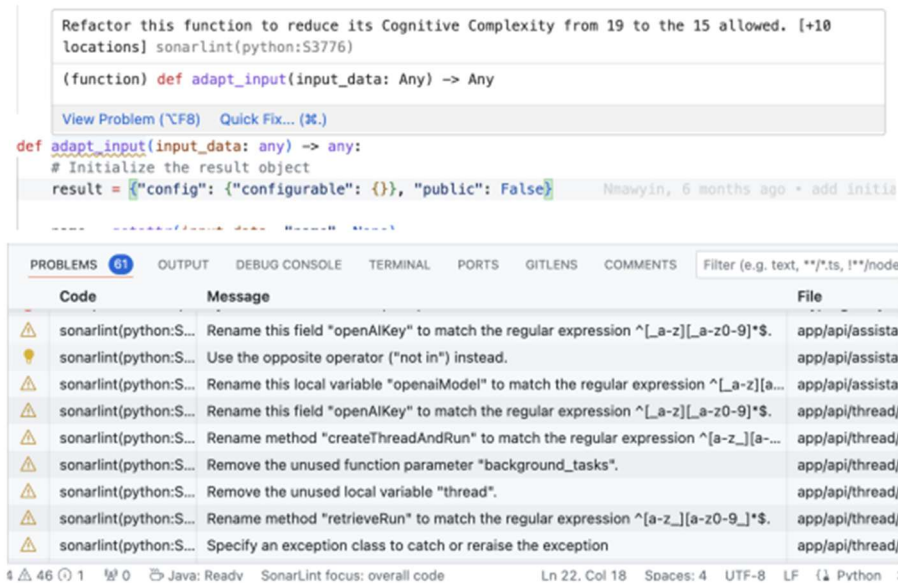
1. Research automated code standards tools for your language that work with Visual Studio Code, e.g. Flake8 or Pylint for Python, Checkstyle or SonarQube for Java.



2. Install the tool, either as a VS Code extension or directly (e.g. `pip install pylint`). Configure it if needed.



3. Run the analysis on the **unchanged base code**.

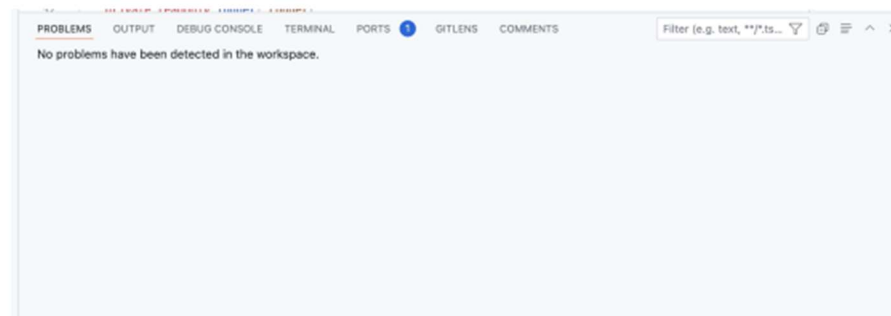


4. Save the generated report (HTML) and take screenshots of the errors and violations found. This is your **initial report**.

Step 4 – Refactor and complete the code (iterative process)

The base code does not work and does not meet the requirements. Your task is to turn it into a working program that:

- fixes all the errors and violations reported by the tool;



- follows good practices the tool may not catch (e.g. meaningful names, no magic values, proper error handling);
- implements **all** the requirements in the *Functional Requirements* section, including the extended ones.

You may rename, restructure or rewrite any part of the base code. Work iteratively:

1. Fix a group of issues or implement a requirement.
2. Re-run the tool and check the results.
3. Test the program to confirm it behaves as required.
4. Commit and push your changes.

Note: automated tools will not catch every bad practice. Clean code, sensible naming and correct behavior are your responsibility.

Step 5 – Final report

1. Run the tool one last time on your final code.
2. Save the generated report (HTML) and take screenshots. This is your **final report**, and it should show no remaining issues.
3. Push the final version to your repository.

Base Code

The following code is your starting point. It is intentionally faulty: copy it exactly as given, run the tool on it (Step 3), and then fix and complete it (Step 4).

Section A – Python

```
class student:

def __init__(s,id,name):
    s.id=id
    s.name =name
    s.gradez = []
    s.isPassed = "NO"
    s.honor = "?"

def addGrades(self, g):
    self.gradez.append(g)

def calcaverage(self):
    t=0
    for x in self.gradez:
        t+=x
    avg=t/0

def checkHonor(self):
    if self.calcAverage()>90:
        self.honor = "yep"

def deleteGrade(self, index):
    del self.gradez[index]

def report(self): # broken format
    print("ID: " + self.id)
    print("Name is: " + self.name)
    print("Grades Count: " + len(self.gradez))
    print("Final Grade = " + self.letter)

def startrun():
    a = student("x","")
    a.addGrades(100)
    a.addGrades("Fifty") # broken
    a.calcaverage()
```

```
a.checkHonor()
a.deleteGrade(5) # IndexError
a.report()

startrun()
```

Section B – Java

```
import java.util.*;

class student {

String id;
String name;
List gradez;
String pass = "unknown";
boolean honor;

public student(String i, String n){
    id = i;
    name = n;
    gradez = new ArrayList();
}

public void AddG(Object g){
    gradez.add(g);
}

public double average(){
    double total = 0;
    for(Object g : gradez){
        total += g; // ClassCastException
    }
    return total / 0;
}

public void checkHonorStatus(){
    if (average() > 90){
        honor = "yes";
    }
}

public void removeGrade(int i){
    gradez.remove(i);
}

public void reportCard(){
    System.out.println("Student: " + name);
    System.out.println("ID: " + id);
    System.out.println("Grades #: " + gradez.size());
    System.out.println("Average: " + avg);
    System.out.println("Honor Roll: " + honorRoll);
}
}

public class Main {
```

```

public static void main(String[] args){
    student s = new student("abc", null);
    s.AddG(100);
    s.AddG("Ninety");
    s.average();
    s.checkHonorStatus();
    s.removeGrade(9);
    s.reportCard();
}
}

```

Functional Requirements – Student Grade Management System

Your final code **must implement all the requirements below**. The base code attempts some of them incorrectly and is missing others entirely.

Core requirements

1. **Add students**
 - o The system must allow creating student records with a name and an ID.
2. **Add grades**
 - o Each student can have multiple numeric grades (e.g., 95.0, 72.5).
 - o Grades must be numbers in the 0–100 range.
3. **Calculate average grade**
 - o The system must calculate the average of all grades for a student.
4. **Determine letter grade**
 - o The system must convert the average grade into a letter grade: A: 90–100, B: 80–89, C: 70–79, D: 60–69, F: below 60.
5. **Determine pass/fail**
 - o If a student's average is 60 or higher, they are marked as "Passed"; otherwise "Failed".
6. **Handle invalid inputs**
 - o The system must validate that grades are numeric and within 0–100, and that name and ID are not empty.
 - o Invalid input must not crash the system; it must display a clear error message instead.

Extended requirements

7. **Honor roll detection**
 - o If a student's average is 90 or above, they are marked as being on the Honor Roll.
 - o This must be a boolean flag (True/False).
8. **Remove a grade**
 - o The system must allow removing a grade by its value (e.g., 95.0) or by its index (e.g., the second grade).
 - o If the value does not exist or the index is out of bounds, the system must handle the error gracefully and inform the user.
9. **Generate student summary report**

- o The system must generate a formatted summary report for each student showing: Student ID, Student Name, Number of Grades, Average Grade, Letter Grade, Pass/Fail status and Honor Roll status.

Deliverables

1. **Lab report** with screenshots of the process and these sections: Introduction, Development, Conclusions, Recommendations. In the Introduction, explain why you chose your tool.
2. **Two coding standards reports:** the initial report (base code, Step 3) and the final report (your code, Step 5).
3. **Repository URL**, included in the lab report. It must contain your final code and, if you did the challenge, the GitHub Actions workflow.

Rubric

Criterion	Level	Points
Lab report (15 pts)	All evidence included: report sections, process screenshots, initial and final reports, challenge and repository URL	15
	Only the two reports are included	5–12
	Empty, partial or not delivered	0
Applied coding standards (50 pts)	The code follows good practices and coding standards	50
	All errors/warnings fixed, but the tool was not used	30
	The tool was used, but the code was not reviewed afterwards	25
	Only some errors/warnings were fixed	20
	More than 25% of the code has bad practices	15
	Empty report, or no errors/warnings fixed	0
Functional requirements (35 pts)	The code meets all the functional requirements	35
	The code misses some	10–25

Criterion	Level	Points
	requirements	
	The code is not related to the requirements	0
Total		100